

Enhancing AlphaEvolve_AgentX for AgentBeats Submission

Context and Goals

AlphaEvolve_AgentX is a self-evolving AI agent framework that generates and iteratively improves code solutions through a feedback loop (Generate → Test → Feedback → Evolve). This architecture (also seen in its successor EvoAgentX) enables autonomous refinement of code by continuously evaluating and optimizing the agent's outputs ¹. The goal is to leverage this foundation to build a **Green (evaluator) agent** for the AgentBeats competition Phase 1. A Green agent in AgentBeats acts as the *environment curator, task provider, and judge* for other AI agents ². In our case, we plan to turn AlphaEvolve_AgentX into a benchmark environment that evaluates **machine-learning code generation tasks** (similar to real Kaggle competitions and research benchmarks). The system will present ML tasks to any Purple (competing) agent and automatically score their performance.

Given the broad, real-world nature of these ML tasks (spanning different datasets and problem domains), the project aligns best with the **OpenEnv Challenge** (custom track) of AgentBeats ³. The OpenEnv track encourages novel, general-purpose environments – which fits our benchmark of diverse ML engineering challenges. (In terms of agent category, this environment tests a **Coding Agent's** abilities in an ML context ⁴, possibly overlapping with *Research Agent* skills when reproducing paper results.) By targeting OpenEnv, we can design a new benchmark from scratch while meeting all required submission components: a public GitHub repo with source and README, a Dockerized Green agent that runs end-to-end, one or more baseline Purple agents (A2A-compatible) for evaluation, registration on the AgentBeats platform with a working leaderboard, evidence of reproducibility, a short abstract, and a demo video ⁵. The following improvement plan outlines how to extend AlphaEvolve_AgentX to meet these goals and the competition's judging criteria (technical robustness, meaningful task design, objective scoring, etc. ⁶).

Architectural Enhancements for ML Workflows

1. Persistent Workspace & State: ML tasks often involve handling datasets and incremental training across multiple steps. We should modify AlphaEvolve's execution sandbox (the `DockerInterpreterToolkit`) to support a **persistent filesystem or volume** that retains data, code, and model checkpoints between agent actions. This is crucial because an agent may need to download a dataset, preprocess it, train a model, then evaluate it in successive turns without losing progress. In frameworks like MLAgentBench, each task environment provides a workspace of files that the agent can read/write repeatedly, producing a final output file for evaluation ⁷. Emulating this, the improved AlphaEvolve environment will maintain state so that an agent's changes to `train.py`, saved models, or intermediate outputs persist throughout the session. This persistence allows iterative refinement of an ML solution (e.g. tuning hyperparameters over multiple attempts) – a pattern essential for real Kaggle-style workflows.

2. GPU and Resource Integration: Many ML tasks (especially deep learning on larger datasets) are computationally intensive. To enable realistic model training and evaluation, the AlphaEvolve Docker environment should be enhanced to allow **GPU acceleration**. Using NVIDIA Docker (or Docker runtime

with GPU support), we can pass through an NVIDIA GPU to the container running the agent's code. The official MLE-bench setup, for example, allocates a GPU and substantial CPU/RAM to each agent run ⁽⁸⁾, underscoring the need for hardware support. By integrating CUDA drivers and libraries in the Docker image and using appropriate Docker compose settings, our Green agent can let Purple agents train neural networks or run GPU-accelerated computations when needed. This ensures that tasks like image classification or large-scale data training are feasible within the evaluation environment. We will also enforce resource and time limits (as MLE-bench does with 24h run caps and set hardware quotas ⁽⁸⁾) to keep evaluations fair and reproducible.

3. Robust Error Handling and Logging: In adapting to ML tasks, the system must handle longer execution times and potential errors (out-of-memory, library import failures, etc.) gracefully. We will improve the interpreter and EvaluatorAgent to capture detailed logs of each code run, including training progress output, errors, and performance metrics. These logs will feed into the feedback loop. If the agent's code crashes or diverges, the Evaluator should catch it and provide a structured error report as feedback. This aligns with competition expectations for robust automation and logging ⁽⁹⁾. Improving logging and sandbox isolation will also aid reproducibility – any run on the same Purple agent and task should yield consistent outcomes, with differences easily debugged via logs.

Integration of ML Benchmarks and Data Sources

1. Leveraging Kaggle Competitions (MLE-bench): Rather than inventing tasks from scratch, we can incorporate existing real-world ML challenges. OpenAI's **MLE-bench** provides a curated set of 75 Kaggle competition tasks covering diverse domains (tabular data, vision, NLP, etc.) ⁽¹⁰⁾. Each task in MLE-bench includes the original competition description, a prepared dataset (with private test split), and reference evaluation code for the competition's metric ⁽¹¹⁾. We will “*agentify*” a subset of these tasks by wrapping them in the AlphaEvolve workflow. Concretely, we plan to add a **KaggleToolkit** (or Kaggle API integration) that can fetch competition data and problem statements. Using MLE-bench's open-sourced scripts ⁽¹¹⁾, the EvaluatorAgent can automatically compute the agent's score on the task (e.g. using the same accuracy or RMSE metric that the real Kaggle used). This gives our Green agent a ground truth scoring method closely aligned with real human benchmarks. Notably, Kaggle leaderboards establish human baselines – e.g. the cutoff scores for bronze, silver, gold medals in each competition ⁽¹²⁾. We will use these baselines to contextualize the agent's performance (see “leaderboard simulation” below). By porting Kaggle tasks into our framework, we ensure the benchmark is **meaningful and diverse** (covering image classification, regression, etc., not just trivial code problems ⁽¹³⁾) and **representative of modern ML engineering challenges** ⁽¹⁴⁾.

2. Incorporating MLAGentBench Tasks: In addition to full Kaggle competitions, we can include more narrowly-scoped ML experimentation tasks as found in Stanford's **MLAgentBench**. MLAGentBench defines 13 tasks that range from improving a baseline model's accuracy on CIFAR-10 by 10%, to solving recent Kaggle-style challenges from 2022-2023, and even some research-oriented tasks ⁽¹⁵⁾. These tasks are designed to run in a shorter time (order of minutes) while still requiring non-trivial ML improvements. We will adapt some of these as evaluation scenarios in AlphaEvolve_AgentX. For example, one task might give the Purple agent a starter `train.py` for a neural network and ask it to achieve a certain accuracy gain, or present a small Kaggle-like problem with provided data. Each task will include: a **task description** (goal and context), **starter code or baseline solution**, and the **dataset** (small enough for quick iteration). By providing starter files and a well-scoped objective (as MLAGentBench does ⁽⁷⁾ ⁽¹⁵⁾), we reduce the chance of the agent flailing and make evaluation more focused. The EvaluatorAgent can directly measure the improvement achieved (e.g. did accuracy increase by the required 10%? Did the new submission beat a baseline score?). Integrating these tasks diversifies our benchmark and tests the agent's ability to *improve existing ML solutions*, a key real-world skill.

3. Reference Paper Implementation Tasks: To push into the **research agent** territory, we can define tasks where the target performance is drawn from academic literature. For instance, an environment could provide a recent ML research paper’s description (or an excerpt of its methodology) and ask the agent to implement the technique and reproduce the reported results. We plan to add a tool (e.g. an **ArxivToolkit** or **ResearchPaperParser**) to fetch key details from a paper – such as the model architecture and the reported metrics on a benchmark dataset. The **EvaluatorAgent** can then compare the agent’s results with the paper’s “golden” results. For example, if a paper reported 95% accuracy on a certain dataset, the agent’s code would be judged on how close it gets to that 95%. This extends our evaluation beyond just Kaggle leaderboards to **state-of-the-art replication**. It addresses the competition guideline of originality and covering gaps in evaluation ¹⁶, by introducing tasks that test an agent’s research comprehension and implementation skill (a capability not covered by many existing benchmarks).

Enhanced Evaluation and Scoring Mechanisms

1. Metric-Based Scoring (Beyond Pass/Fail): In the current AlphaEvolve setup, code may be evaluated by simple tests or success/fail criteria. We will upgrade the **EvaluatorAgent** to return rich **quantitative metrics** for each task run. Depending on the task, this could be accuracy, F1-score, RMSE, AUC, log-loss, execution time, etc. Adopting the exact metrics used in the original competition or dataset ensures the scores are meaningful. For example, for a Kaggle classification task, the evaluator will compute the agent’s prediction accuracy on a hidden test set and return that value. For a model-training task, it might output the final model’s accuracy and training time. These numeric scores allow us to implement **nuanced grading** as recommended by the competition – not just whether the agent solved the task, but *how well* (what score) and *how efficiently* ¹⁷. We will also support partial credit: tasks can have thresholds for different achievement levels (e.g. 90% accuracy might be a “pass”, 95% a “high score”). This multi-level evaluation is more informative than binary pass/fail ¹⁷ and will be reflected on the AgentBeats leaderboard as well.

2. Automated Leaderboard and Medal Benchmarks: To contextualize agent performance, our Green agent will include a **leaderboard simulation**. Using historical human results from Kaggle, we know the score ranges for bronze, silver, and gold medals for each competition ¹². After each evaluation run, we can rank the Purple agent’s score relative to these benchmarks. For example, if the agent’s model achieved an AUC that would place it in the top 10% of the original competition, we label that as a “Bronze-level performance” – and if it’s near the top 1%, that’s “Gold-level”. OpenAI’s MLE-bench uses this approach: they reported that their best agent (with an advanced scaffold) obtained at least a bronze medal on ~16.9% of the 75 competitions ¹⁸. We will incorporate such statistics: the **EvaluatorAgent** can output not only raw metrics but also a comparison like “Result is equivalent to a Kaggle Silver (top 5%) on this task.” This provides an intuitive measure of the agent’s prowess and captures multiple dimensions of performance (effectiveness and how it stacks up to human experts). The AgentBeats platform allows each Green agent to host a leaderboard – we will ensure our leaderboard displays these relative achievements, and perhaps aggregate an overall “medal count” for each Purple agent. This competitive framing can drive innovation and aligns with the spirit of AgentBeats’ unified benchmarking ¹⁹ ²⁰.

3. Integrated Safety and Efficiency Checks: Although our focus isn’t the security track, we will integrate basic safety checks in evaluation – for example, monitoring for the Purple agent trying to access the internet or forbidden resources if the task doesn’t allow it, or penalizing overly inefficient solutions (e.g., an agent that brute-forces and times out). The judging criteria emphasize capturing multiple performance dimensions including efficiency and safety ¹⁷. Thus, our **EvaluatorAgent** can deduct points or mark a task as failed if the agent violates constraints (like using disallowed libraries or

exceeding memory limits). We will document these rules clearly in the README and abstract, so that participants know the boundaries of the environment.

4. Feedback for Self-Evolution: Since AlphaEvolve_AgentX is inherently a self-improving system, we will use the Evaluator's outputs to create structured feedback for the Purple agent under test. For instance, after an agent's code is executed, the EvaluatorAgent will provide a summary: *"Score = 0.85 AUC, which is below the 0.90 target. The model is underfitting the data (validation loss still high). Consider adding more features or increasing model complexity."* This kind of hint can be generated by an LLM (prompted with the error logs and metric shortfall) and would be fed back if we allow iterative attempts. While for competition scoring we typically run a fixed agent, having this feedback mechanism is great for the baseline agent development and showcases the *self-evolution* aspect. It demonstrates an innovative evaluator that not only scores but also guides improvement – a feature that could score creativity points in design ¹⁶.

Implementation Roadmap and Example Workflow

To realize the above enhancements, we propose the following development steps:

- 1. Define ML Task Specifications:** Extend the framework to have a `MLTask` configuration object for each task/benchmark. This would include fields like `dataset_url` or `ID`, `task_description`, `evaluation_metric` (e.g. "accuracy", "RMSE"), and possibly a `baseline_solution` or baseline score. For instance, one `MLTask` could point to the Titanic survival prediction competition, with `metric = Accuracy` and `baseline_score = 0.78` (historical median). Another could be "Increase CIFAR-10 CNN accuracy by 10%", specifying the starting code and required improvement. Having a standardized spec makes it easy to add new tasks and ensures the Green agent can enumerate them for the Purple agent.
- 2. Integrate Kaggle Data and API:** Implement a `KaggleToolkit` using Kaggle's public API (or the official Kaggle Python library) to programmatically download competition datasets and maybe even submit predictions to Kaggle if needed. In offline mode, the toolkit can retrieve datasets and store them in the persistent workspace for the agent to use. We will also include any **data preprocessing utilities** needed (for example, Kaggle data often comes as CSVs or images – we might provide a helper to load it into Python). Additionally, if using MLE-bench's prepared data splits and evaluation scripts, we will bundle those into the environment. For each task, the EvaluatorAgent will call the appropriate **grading script** to get the score, ensuring consistency with the original metric ¹¹. This step involves careful Docker setup to include necessary ML libraries (TensorFlow/PyTorch, scikit-learn, etc.) and the Kaggle API credentials (handled securely via environment variables).
- 3. Implement Iterative Evaluation Loop:** With tasks and data in place, we enhance the core AlphaEvolve loop to support multiple attempts on a single task:
4. The Purple agent (e.g. AlphaEvolve's built-in agent or another baseline agent) is given the task description and the initial starter code or hints. It then **generates an initial solution** – for example, writing a Python training script in the persistent workspace.
5. The **EvaluatorAgent (Green)** executes the agent's code inside the Docker sandbox, monitors the run (capturing output logs), and computes the performance metric. Suppose the agent's first attempt yields an accuracy of 80% when the target was 90%.
6. If the result does not meet the success criteria or can be improved, the EvaluatorAgent uses the outcome to craft **feedback**. For instance, it might note: *"Accuracy is 10% below target; training logs*

suggest overfitting. The agent's model may need regularization." This feedback is returned to the Purple agent along with any error traces.

7. The Purple agent can then **"evolve" its solution** using this feedback – for example, it might modify the code to add dropout layers or try a different algorithm. This generate-test-feedback cycle repeats (within a limit of iterations or time) until the agent either meets the goal or exhausts its tries. Each iteration's score is recorded.
8. Finally, the best result achieved by the agent is taken as the outcome for that task. The Green agent will output the final score and a transcript of the agent's iterations (this helps with reproducibility and analysis).
9. **Baseline Purple Agent Setup:** We will prepare at least one baseline Purple agent to demonstrate our benchmark. Ideally, we can use the existing AlphaEvolve agent (the one that performs self-evolution) as a contestant by configuring it to interface via the AgentBeats A2A protocol. This agent, powered by an LLM, would attempt the tasks using the iterative approach above. If AlphaEvolve_AgentX's built-in agent isn't directly suited for ML coding, we may adopt an alternative baseline. One strong candidate is OpenAI's **AIDE scaffold**, which was shown to reach a bronze medal in ~17% of Kaggle tasks with a GPT-based planner ¹⁸. Another option is a simplified AutoGPT-like agent specialized for ML (similar to what MAgentBench uses with a step-by-step planner ²¹). We will ensure the baseline agent is **A2A-compatible** (following AgentBeats protocols for messaging). This baseline is mainly to validate that our Green agent works end-to-end and to provide an initial score on the leaderboard. Over time, others can plug in more advanced Purple agents to compete.
10. **Testing & Reproducibility:** We'll run multiple evaluation cycles with the same configuration to ensure consistent results, addressing the competition's reproducibility criterion ⁹. Because stochastic LLM decisions could lead to variability, our EvaluatorAgent might use fixed random seeds for any data splits and possibly run each agent task multiple times to record an average performance (similar to MLE-bench's multi-seed approach ²²). We will document how to reproduce each task's evaluation in the README, and provide scripts to launch the Docker container and run the entire Green-vs-Purple interaction automatically. The Docker image will encapsulate all dependencies (ML frameworks, data fetching tools, etc.), so that judges or other participants can easily rerun our benchmark on their machines or the AgentBeats platform.

Conclusion and Track Fit

By implementing these improvements, **AlphaEvolve_AgentX** will transform into a comprehensive benchmark agent that challenges ML-focused AI agents in realistic coding tasks. It will define rich environments ranging from Kaggle competitions to cutting-edge research reproductions, with automated and nuanced scoring to push agents beyond trivial outputs. This directly addresses the AgentBeats mandate of creating *"fully automated evaluation systems"* as Green agents ². In terms of competition tracks, our project is best positioned in the **OpenEnv Challenge**, which seeks state-of-the-art new environments ²³. The lack of a specific domain constraint ("no specific evaluation domain") and the emphasis on diverse ML tasks make OpenEnv the ideal track for this submission.

In summary, **the improved AlphaEvolve_AgentX will serve as a novel benchmark for AI coding agents**, evaluating their ability to handle end-to-end machine learning problems. We have aligned the design with all competition requirements (from a Dockerized implementation to reproducibility and documentation) and incorporated feedback from recent research benchmarks to ensure its relevance. By using persistent workspaces, integrating real-world ML challenges, supporting rich metrics and

iterative feedback, and ensuring alignment with human baselines, this Green agent will not only be a strong AgentBeats submission but also a valuable tool for advancing autonomous ML agent research.

Proposed New Tooling Summary: To support the above features, we will introduce several modular tools: - **KaggleToolkit** – for data fetching and interaction with Kaggle or offline competition data (leveraging OpenAI’s MLE-bench resources for consistency in evaluation) ¹¹. - **ResearchPaperParser (ArxivTool)** – to retrieve target metrics or pseudocode from research papers, enabling tasks that compare an agent’s output to published results. - **PerformanceEvaluator Module** – a flexible evaluator that computes domain-specific metrics (classification scores, regression error, etc.), checks against thresholds, and formats leaderboard rankings (e.g. medal categories ¹²). - **CudaDockerToolkit** – extensions to the existing Docker tool to mount persistent volumes and expose GPUs, aligning the agent’s execution environment with standard ML workflow needs (e.g. similar to containers used in MLE-bench with GPU support ⁸).

By integrating these components into the AlphaEvolve_AgentX framework, we ensure our submission is **comprehensive, innovative, and competition-ready** – demonstrating a thorough improvement plan that bridges the gap between autonomous code generation agents and real-world machine learning engineering challenges. ¹⁷ ¹²

¹ GitHub - EvoAgentX/EvoAgentX: EvoAgentX: Building a Self-Evolving Ecosystem of AI Agents
<https://github.com/EvoAgentX/EvoAgentX>

² ³ ⁴ ⁵ ⁶ ⁹ ¹⁶ ¹⁷ ¹⁹ ²⁰ ²³ AgentX AgentBeats Competition
<https://rdi.berkeley.edu/agentx-agentbeats.html>

⁷ ¹⁵ ²¹ MLEAgentBench: Evaluating Language Agents on Machine Learning Experimentation
<https://arxiv.org/pdf/2310.03302>

⁸ ¹⁰ ¹¹ ¹² ¹³ ¹⁴ ¹⁸ ²² MLE-bench: Autonomous ML Engineering Benchmark
<https://www.emergentmind.com/topics/mle-bench>